Roots That Are Always There

The virtual roots of a real polynomial, machine-checked in Lean 4

Carles Marín*

Independent researcher

karlesmarin@gmail.com

June 2026

Abstract

A real polynomial of degree d may have fewer than d real roots, but it always has exactly d *virtual roots*: continuous, semialgebraic substitutes $\rho_{d,1}(p) \le \cdots \le \rho_{d,d}(p)$ that coincide with the real roots when these exist and otherwise sit where the polynomial comes closest to vanishing. They were introduced by González-Vega, Lombardi and Mahé [1] and are the natural carrier of the Budan–Fourier count [2]. I report a complete, `sorry`-free formalization in Lean 4 over Mathlib of their *structural theory*: the recursive construction by a choice operator $\mathcal{R}_d$ that minimises $|p|$ on each interval where p' keeps a constant sign; the fundamental count, that there are exactly d of them (`vroots_length`); that they come out sorted and inside the bracketing interval (`vroots_isChain`, `vroots_subset_Icc`); that $\mathcal{R}_d$ returns an actual root wherever p has one (`R_eval_eq_zero_of_le_zero_le`); and the *Rolle interlacing*

$$\rho_{d,r}(p) \ \le\ \rho_{d-1,r}(p') \ \le\ \rho_{d,r+1}(p),$$

the statement that each virtual root of p' is wedged between two consecutive virtual roots of p (`vroots_interlacing`). To the best of my knowledge—based on searches in June 2026 of Mathlib, the Coq `mathcomp-real-closed` library, the Isabelle AFP, and HOL Light (Section 8)—virtual roots had not been formalized in any interactive theorem prover; this is their first development. What is deliberately *not* here is the exact Budan–Fourier count, that the number of virtual roots in $(a, b]$ equals the sign-variation drop $\mathrm{V}(a) - \mathrm{V}(b)$: that theorem fuses this construction with a second, independent one (the `fourierVar` of the companion Budan–Fourier formalization [18]) and is the explicit sequel, named honestly in Section 8. Every theorem here depends only on the three standard axioms `propext`, `Classical.choice`, `Quot.sound`.

The development is a single Lean file, `VirtualRoots.lean`; its load-bearing lemmas are gathered in Section 7, and its limits—chiefly the deferred exact count—are stated in Section 8. If you read only two things, read the interlacing (Theorem 1) and Figure 1.

1 A count that does not wobble

How many roots does a polynomial have? The honest answer wobbles. A real polynomial of degree d has *somewhere between* 0 *and* d real roots, and the number lurches as the coefficients move: $x^2 - t$

*Formalized with the assistance of an AI system (Claude, Anthropic); the mathematics is classical, and every claim, scope statement, and limitation is the author's responsibility. The boundary of the contribution—in particular what is *not* yet formalized—is set out plainly in Section 8, and the Lean kernel, not the assistant, is what certifies the proofs.

has two real roots, then one, then none as t falls through zero, while its degree never changes. A count that jumps when nothing essential has is a poor count. **There is a better one: a degree-d polynomial always has exactly d *virtual roots***—real numbers $\rho_{d,1}(p) \le \cdots \le \rho_{d,d}(p)$ that coincide with the genuine roots when those exist and otherwise sit where the polynomial comes closest to vanishing. They never jump, never disappear, and vary continuously with the coefficients. This paper machine-checks the structural theory of that better count.

The picture to keep is the simplest non-trivial one. The polynomial $p = x^2 + 1$ has no real root, yet its graph dips to a single closest approach at $x = 0$. Virtual roots see that approach: p has the double virtual root $\rho_{2,1} = \rho_{2,2} = 0$ (Figure 1). Where a real root would be, there is a virtual one; where two conjugate roots hide, the two virtual roots collide at the foot of the dip rather than vanish. Nothing is invented—the construction is explicit and semialgebraic—and nothing is lost: **the count is always** d. The driving thread of the paper is this contrast, that real roots are a count that wobbles and virtual roots are the stable d-element family underneath, and that the stability is not asserted but *constructed*, concretely enough for a kernel to check.

The object is due to González-Vega, Lombardi and Mahé [1], and it is the natural carrier of the Budan–Fourier count, made an equality through it by Coste, Lajous, Lombardi and Roy [2]. This paper formalizes the part of that story that is structural—the construction, the count, the order, and the Rolle interlacing—and is honest, in Section 8, about the arithmetic count it leaves to the sequel. I came to it as I came to Sturm [19] and Budan–Fourier [18]: by asking which classical results in real-root counting a mature library is still missing, and finding the virtual roots absent from every prover I could search.

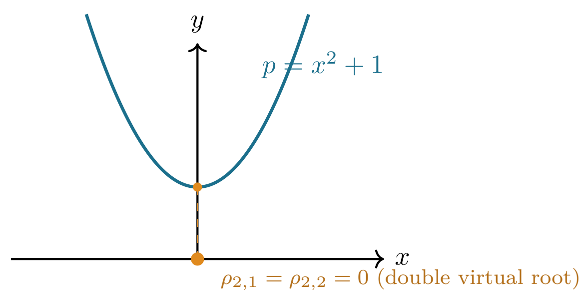

Figure 1: Roots that are always there. The polynomial $x^2 + 1$ has *no* real root, but a double virtual root at $x = 0$, where the graph comes closest to the axis. A degree-2 polynomial has two virtual roots whatever its discriminant; here they coincide. In Lean, on a bracketing interval, the construction returns the list `[0, 0]`.

2 The d roots that are always there

The idea that a polynomial carries d canonical real "roots" regardless of how many are genuine is González-Vega–Lombardi–Mahé's [1], set in the sign-variation theory whose textbook account is Basu–Pollack–Roy [11], Ch. 2. The definition is `vroots` and the count is `vroots_length`.

Between two consecutive critical points of p—two consecutive roots of p'—the derivative keeps a constant sign, so p is strictly monotone there and $|p|$ has a single minimum. The point where that minimum is attained is a virtual root: the unique root of p in the cell if p changes sign across it, and otherwise the cell endpoint where p is smallest. The critical points of p are the virtual roots of p', so the construction recurses on the derivative, exactly as Rolle's theorem [6] interleaves the roots of p and p'.

Why exactly d. A degree-d polynomial has a degree-$(d-1)$ derivative, hence—by induction—$d-1$ virtual roots of p'; these cut the line into d cells, and each cell contributes one virtual root of p. The count never degenerates: it is the degree, not the number of real roots. This is the first theorem, and the cleanest:

$$\boxed{\#\{\text{virtual roots of } p\} \;=\; \deg p.}$$

In Lean this is `vroots_length`: the list `vroots lo hi p` of virtual roots has length `p.natDegree`. Real roots are at most d (and can be far fewer, or none); **virtual roots are always exactly** d. The gap between the two counts is precisely the even Budan–Fourier surplus, now realized as collisions of virtual roots away from the axis.

The recipe is clean on paper—*minimise* $|p|$ *on each* p'*-monotone cell*—but "the point where $|p|$ is least" is a choice, and "the cells" are cut by an object defined by the same recursion. Both have to be made explicit before a kernel will accept them.

3 Building them: the choice operator $\mathcal{R}_d$

The recursive construction—cut the line by the critical points, minimise $|p|$ on each piece—is the definition of González-Vega–Lombardi–Mahé [1], tied to the sign-variation count by Coste [2]. Where a paper proof says "take the point where $|p|$ is least," a formal one must produce it: this is the operator `R` (existence `exists_isMinOn_abs_eval`), assembled by the recursion `vroots`.

The cells of p are cut by the virtual roots of p' together with two outer fences. Rather than work on all of $\mathbb{R}$ from the start, I fix a closed interval $[lo, hi]$ and build the virtual roots inside it; taking lo, hi beyond every root of the derivative tower (a Cauchy bound) recovers the global object, and that last step is discussed in Section 8. Working on $[lo, hi]$ keeps every quantity a genuine real number and lets the recursion reuse one choice operator verbatim.

The choice operator $\mathcal{R}_d$. On a closed interval $[a, b]$, the continuous function $|p|$ attains a minimum (the interval is compact); call a minimiser $\mathcal{R}_d(a, b, p)$. In Lean this is `R`, defined by choosing a minimiser whose existence is `exists_isMinOn_abs_eval`. Two facts about it carry the whole theory. First, it **lands in the interval**: `R_mem` says $\mathcal{R}_d(a, b, p) \in [a, b]$. Second, it **captures actual roots**: if $p(a) \le 0 \le p(b)$ (or the reverse), the intermediate value theorem gives a root, the minimum of $|p|$ is then 0, and the minimiser is that root—`R_eval_eq_zero_of_le_zero_le`. So on a cell where p changes sign, $\mathcal{R}_d$ is the genuine root; where p does not, it is the nearer endpoint. That is the precise sense in which virtual roots extend the real ones.

The choice operator $\mathcal{R}_d(a, b, p)$ (recipe)

On a closed interval $[a, b]$, take **any minimiser of** $|p|$ (it exists, $[a, b]$ is compact). Two facts are all the theory needs. **It lands in the interval**, $\mathcal{R}_d(a, b, p) \in [a, b]$ (`R_mem`); this single fact yields the interlacing. **It is an actual root when p has one**: if p changes sign across $[a, b]$, the minimum of $|p|$ is 0, so $\mathcal{R}_d$ is a genuine root (`R_eval_eq_zero_of_le_zero_le`); this is how virtual roots extend the real ones. Uniqueness, where p' keeps a sign, comes free from the monotonicity bricks of Q3—so the *chosen* minimiser is in fact the only one.

The recursion. The virtual roots are then assembled by listing the breakpoints

$$\text{bps} \;=\; lo \;::\; (\text{virtual roots of } p') \;+\!\!+\; [\,hi\,],$$

and applying $\mathcal{R}_d$ to each consecutive pair: `vroots` $lo\, hi\, p =$ `zipWith` $(\mathcal{R}_d\, p)$ bps bps.tail. A degree-0 polynomial has no virtual roots; otherwise the recursion descends to p', terminating because $\deg p' < \deg p$ (`natDegree_derivative_lt`). The well-founded definition and the length theorem are the content of `vroots` and `vroots_length` (Figure 2).

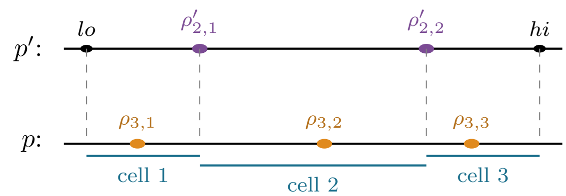

Figure 2: The recursion, for $\deg p = 3$. The two virtual roots of p' (plum) together with the fences lo, hi cut $[lo, hi]$ into three cells; on each cell p' keeps a constant sign, so $\mathcal{R}_d$ returns one virtual root of p (amber). Each $\rho_{3,j}$ lies in its own cell—this is what makes them come out sorted and interlaced with the $\rho'_{2,j}$.

4 Making the construction formal

The construction is classical and the interlacing has a one-line paper proof—"$\mathcal{R}_d$ lands in its cell, and cells share endpoints"—so what costs the work is not new mathematics but the bookkeeping that makes a recursive, choice-defined family pass a kernel, the analogue of the list-surgery the companion Budan–Fourier development [18] needed. It enters in `vroots_chain_mem` (the master invariant) and `isChain_zipWith_R`.

A noncomputable choice, used honestly. $\mathcal{R}_d$ is a *chosen* minimiser, so `R` is `noncomputable` and the development leans on `Classical.choice`. That is sound and standard, but it means the virtual roots are specified, not computed: the theorems are about *a* minimiser, and what makes them unambiguous is monotonicity. The two *monotonicity bricks* shared with the companion Budan–Fourier work—`eval_strictMonoOn_of_derivative_pos` and its negative twin—say that where p' keeps a constant nonzero sign, p is strictly monotone, so $|p|$ has a *single* well and the minimiser is unique. The formal development needs only existence and the two facts `R_mem` / `R_eval_eq_zero`; uniqueness is what licenses the informal picture.

The breakpoints are a chain, and that is the crux. Everything downstream rests on one invariant: the augmented breakpoint list $lo :: ($`vroots` $lo\, hi\, p')$ $+\!+ [hi]$ is *sorted*, and every virtual root lies in $[lo, hi]$. The two halves are mutually dependent—a list is sorted only if its members are in range, and $\mathcal{R}_d$ lands in range only between sorted breakpoints—so they are proved together, by a single induction down the derivative tower (`vroots_chain_mem`). The sortedness is the engine: because $\mathcal{R}_d(\mathrm{bps}[r], \mathrm{bps}[r+1])$ lies in $[\mathrm{bps}[r], \mathrm{bps}[r+1]]$ and consecutive breakpoints are ordered, consecutive virtual roots are ordered too (`isChain_zipWith_R`). This is Rolle's theorem in list form, and it is the part the machine would not let me wave away.

Indexing, and the small traps. Turning "each virtual root sits between its breakpoints" into a statement about `getElem` required care that a hand proof hides. Rewriting a proof-carrying indexed access (`l[r]` drags a proof $r <$ `l.length`) is not a plain `rw`; it goes through `simp` with the index lemmas, relying on proof irrelevance to reconcile the bounds. And a destructuring `rcases` on an equation $w = lo$ silently *substitutes* lo away in that branch—a one-character fix (`le_rfl` for `le_refl lo`), but the kind of thing only a kernel catches. These are not deep, but they are exactly the hand-waves a proof assistant refuses.

The chain and the per-index localization, once proved, are exactly what the interlacing rests on.

5 The Rolle interlacing

That the roots of p and p' interleave is Rolle's, from 1690 [6]; that the *virtual* roots interleave the same way—even when no real roots exist—is the González-Vega–Lombardi–Mahé refinement [1] and the structural core of Coste's count theorem [2]. It is `vroots_interlacing`, resting on the per-index localization `vroots_getElem_mem`.

The interlacing of the virtual roots of p with those of p':

Theorem 1 (Rolle interlacing of virtual roots; `vroots_interlacing`). *For $lo \le hi$ and each index r in range,*

$$\rho_{d,r}(p) \;\le\; \rho_{d-1,r}(p') \;\le\; \rho_{d,r+1}(p),$$

i.e. `(vroots lo hi p)[r]` $\le$ `(vroots lo hi (derivative p))[r]` $\le$ `(vroots lo hi p)[r+1]`.

The proof is two applications of the localization `vroots_getElem_mem`—each virtual root of p lies between two breakpoints—together with the identity that the interior breakpoints *are* the virtual roots of p'. It is the discrete shadow of Rolle's theorem, and it places the virtual-root family on the same footing as the interlacing families that run through the real-rootedness literature—Newton's inequalities, the Heilmann–Lieb matching polynomial—where a sequence and its derivative interleave. That this now holds for a family defined even when no real roots exist is the point.

What it is the road to. The interlacing is the structural half of the Budan–Fourier story. The arithmetic half—that the number of virtual roots in a half-open interval $(a, b]$ equals the sign-variation drop $\mathrm{V}(a) - \mathrm{V}(b)$ of the derivative tower [2]—is the equality that the classical Budan–Fourier inequality [18] only bounds. It is deliberately not proved here, for an honest reason explained next: it fuses this construction with a second, independently recursive one. The structural theory is complete; the count is the sequel.

6 What a certified virtual-root theory enables

A machine-checked virtual-root family is the missing middle term between two things Mathlib already has and one it does not. It has Descartes' rule (the coefficient sign count) and, in the companion work, the Budan–Fourier *inequality*; it does not have the object that would make that inequality an equality. Virtual roots are that object. Downstream, they are the right input to a certified real-root *isolation* procedure in the line of Vincent, Collins and Akritas [10, 9, 8] and the sign-variation theory of Basu–Pollack–Roy [11]—the count is exact, so a subdivision method reading $\mathrm{V}(a) - \mathrm{V}(b)$ on each half has, in principle, a verified guarantee once the count theorem is added. And because the family is defined for every polynomial, real-rooted or not, it connects the interval methods to the algebra of the discriminant: a pair of virtual roots collides exactly when two real roots are about to become complex, which is where the even Budan–Fourier surplus comes from (Figure 3). The structural theory formalized here is the foundation all of that stands on.

7 The building blocks

Table 1 gathers the load-bearing results, all `sorry`-free in `VirtualRoots.lean`; Table 2 measures the development. The dependency is short: the monotonicity bricks give the existence and the two properties of $\mathcal{R}_d$; the combined induction `vroots_chain_mem` proves sortedness and range together; and the per-index localization gives the interlacing.

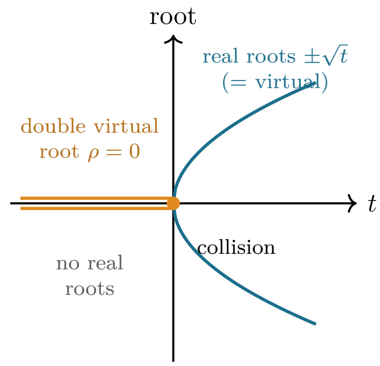

Figure 3: Roots that are always there, as a parameter moves. For $p_t = x^2 - t$ the two real roots $\pm\sqrt{t}$ (teal) exist only for $t \geq 0$ and vanish as t drops below 0; the two *virtual* roots (amber) persist for *all* t, coinciding with the real roots when $t \geq 0$ and colliding into the double virtual root $\rho = 0$ for $t < 0$. The real-root count jumps $2 \to 0$ at $t = 0$; the virtual-root count stays 2. The collision at $t = 0$ is exactly where a conjugate pair forms—the source of the even Budan–Fourier surplus. (A different view of the same phenomenon as Figure 1, now in motion.)

Lean name	Statement	Status
`R_mem`	$\mathcal{R}_d(a, b, p) \in [a, b]$	proved
`R_eval_eq_zero_of_le_zero_le`	$p(a) \leq 0 \leq p(b) \Rightarrow p(\mathcal{R}_d(a, b, p)) = 0$	proved
`vroots_length`	$\#\{\text{virtual roots of } p\} = \deg p$	proved
`vroots_isChain`	$lo :: (\text{vroots } p) +\!\!+ [hi]$ is sorted	proved
`vroots_subset_Icc`	every virtual root lies in $[lo, hi]$	proved
`vroots_getElem_mem`	$\text{bps}[r] \leq \rho_{d,r}(p) \leq \text{bps}[r+1]$	proved
`vroots_interlacing`	$\rho_{d,r}(p) \leq \rho_{d-1,r}(p') \leq \rho_{d,r+1}(p)$	proved
`card_virtualRoot_eq_fourierVar`	$\#$ in $(a, b] = \mathrm{V}(a) - \mathrm{V}(b)$	*future (§8)*

Table 1: The structural theory of virtual roots, all `sorry`-free in `VirtualRoots.lean` (Lean 4 / Mathlib). `#print axioms` on `vroots_interlacing`, `vroots_isChain`, `vroots_getElem_mem` reports only `propext`, `Classical.choice`, `Quot.sound`. The last row is the named sequel, not part of this development.

Quantity	Value
Lines in `VirtualRoots.lean`	369
Public theorems / definitions (+ private)	16 / 2 (+1)
Mathlib dependencies (imports)	5 modules
`Deriv.Polynomial`, `Deriv.MeanValue`, `Topology.Algebra.Polynomial`, `Topology.Order.Compact`, `Topology.Order.IntermediateValue`	
`sorry` / `admit` / `native_decide`	0
Axioms beyond Lean's core	0 (only `propext`, `Classical.choice`, `Quot.sound`)
Compile time (warm Mathlib)	$\approx 8\,\text{s}$
Toolchain	Lean 4 (godsil pin v4.30) over Mathlib

Table 2: The development measured. Build: `lake env lean VirtualRoots.lean`, exit 0; compile time is for checking the file against a prebuilt Mathlib. The footprint is deliberately small—five analysis/topology modules and no heavy algebra—because the construction is elementary.

8 The boundary of the contribution

What is proved. The structural theory of the $\mathcal{R}_d$ virtual roots, sorry-free: existence and the count ($= \deg p$), that they are sorted and lie in the bracketing interval, that $\mathcal{R}_d$ returns an actual root wherever p changes sign, and the Rolle interlacing. These are the results of González-Vega–Lombardi–Mahé [1] in their structural part.

What is not proved, and is the explicit sequel. The exact Budan–Fourier count—that the number of virtual roots in $(a, b]$ equals $\mathrm{V}(a) - \mathrm{V}(b)$ [2]—is *not* formalized here. It is not a missing line; it fuses two independent recursive constructions, the $\mathcal{R}_d$ minimiser of this file and the sign-variation function `fourierVar` of the Budan–Fourier formalization [18], and it is a development in its own right. Stating it as done would be dishonest; it is named here as the next milestone, with the two endpoints (the interlacing here, the inequality there) now both in place.

Scope of the construction. The virtual roots are built on a fixed bracketing interval $[lo, hi]$. The global object on all of $\mathbb{R}$ is recovered by taking lo, hi beyond every root of the derivative tower—a Cauchy bound, available in Mathlib (`Polynomial.cauchyBound`)—but that extension is not carried out in this file.

Novelty. To the best of my knowledge, virtual roots had not been formalized in any interactive theorem prover before this. The claim rests on searches in June 2026 of: Mathlib (no virtual roots; it has Descartes via `Polynomial.signVariations`); the Coq `mathcomp-real-closed` library (actual roots and the Thom encoding, not virtual roots); the Isabelle AFP, whose `Budan_Fourier` entry formalizes the inequality with multiplicity but not the virtual-root object; and HOL Light (Sturm and Descartes, not virtual roots). I make this claim at the level of the named object; I cannot exclude an unpublished or differently-named development, and I would welcome a pointer.

Authorship. The mathematics is classical, due to the authors cited. The formalization was carried out in close interaction with an AI assistant (Claude, Anthropic): the choice of target, the overall proof architecture, the statements, and the scope and novelty claims are mine; the assistant drafted and iterated much of the Lean under that direction. The division does not bear on validity—a proof is accepted only when the Lean kernel type-checks it, which it does, with the axiom profile reported above—but it is stated plainly here for scientific attribution.

9 What remains open

Three facts this development does not settle mark where it points next; each is a place the work stops short, and each is the start of the following one.

1. *From two constructions to one count.* The headline equality $\#\{\text{virtual roots in } (a, b]\} = \mathrm{V}(a) - \mathrm{V}(b)$ is the sequel, and there are two honest routes to it. One *bridges* the construction here—the $\mathcal{R}_d$ minimiser—to the sign-variation function `fourierVar` of the companion work [18], turning that paper's inequality [2, 1] into an equality. The other *defines* virtual roots a second time, as the jumps of `fourierVar`, where the count is almost definitional, and then proves the two definitions agree. The interlacing proved here is free in the first definition; the count is free in the second; **the equivalence of the two definitions is the real theorem**. Which route is shorter in Lean is itself the question.

2. *Continuity, the other half of their reason for being.* Virtual roots vary *continuously* with the coefficients—this is half of why González-Vega–Lombardi–Mahé introduced them [1]—and continuity is not touched here. A formalization would have to make $\mathcal{R}_d$, a chosen minimiser, depend continuously on p; the monotonicity bricks that pin the minimiser down are the natural starting point, but the statement is genuinely analytic.

3. *One interlacing, three faces.* The Rolle interlacing places virtual roots beside the families that run through the real-rootedness literature: the roots of a polynomial and its derivative (classical Rolle [6]), Newton's inequalities [20], and the Heilmann–Lieb matching polynomial. Eisermann's abstract *Sturm chains* [12] already axiomatize one such interlacing structure. Is there a single Lean notion of "interlacing family" that subsumes all of these—and where does the virtual-root family, defined even when no real roots exist, sit inside it? That is the bead I would most like to carry forward.

References

[1] L. González-Vega, H. Lombardi, L. Mahé, Virtual roots of real polynomials, *J. Pure Appl. Algebra* **124** (1998), no. 1–3, 147–166.

[2] M. Coste, T. Lajous-Loaeza, H. Lombardi, M.-F. Roy, Generalized Budan–Fourier theorem and virtual roots, *J. Complexity* **21** (2005), no. 4, 479–486.

[3] R. Descartes, *La Géométrie* (1637); the rule of signs bounds the number of positive roots by the number of sign changes of the coefficients.

[4] F. D. Budan de Boislaurent, *Nouvelle méthode pour la résolution des équations numériques*, Paris (1807; 2nd ed. 1822).

[5] J. Fourier, *Analyse des équations déterminées*, Firmin Didot, Paris (posthumous, 1831).

[6] M. Rolle, *Traité d'algèbre* (1690); between two consecutive real roots of a differentiable function lies a root of its derivative.

[7] C. Sturm, Mémoire sur la résolution des équations numériques, *Mémoires présentés par divers savants à l'Académie Royale des Sciences* **6** (1835) 271–318.

[8] A. G. Akritas, Reflections on a pair of theorems by Budan and Fourier, *Mathematics Magazine* **55** (1982) 292–298.

[9] G. E. Collins, A. G. Akritas, Polynomial real root isolation using Descartes' rule of signs, in *Proc. SYMSAC '76*, ACM, 1976, 272–275.

[10] A. Alesina, M. Galuzzi, A new proof of Vincent's theorem, *L'Enseignement Mathématique* **44** (1998) 219–256.

[11] S. Basu, R. Pollack, M.-F. Roy, *Algorithms in Real Algebraic Geometry*, 2nd ed., Algorithms and Computation in Mathematics **10**, Springer, 2006 (sign-variation theory, Ch. 2, and virtual roots).

[12] M. Eisermann, The fundamental theorem of algebra made effective: an elementary real-algebraic proof via Sturm chains, *Amer. Math. Monthly* **119** (2012), no. 9, 715–752 (abstract "Sturm chains" and the Cauchy index).

[13] W. Li, L. C. Paulson, Counting polynomial roots in Isabelle/HOL: a formal proof of the Budan–Fourier theorem, in *Proc. CPP 2019*, ACM, 2019, 52–64; Archive of Formal Proofs entry "The Budan–Fourier Theorem" (W. Li, 2018), `https://isa-afp.org/entries/Budan_Fourier.html` (formalizes the inequality with multiplicity, not the virtual-root object).

[14] C. Cohen, Construction of real algebraic numbers in Coq, in *Interactive Theorem Proving (ITP 2012)*, LNCS **7406**, Springer, 2012, 67–82 (basis of `mathcomp-real-closed`: actual roots and the Thom encoding, not virtual roots).

[15] M. Eberl, Sturm's Theorem, *Archive of Formal Proofs* (2014), `https://isa-afp.org/entries/Sturm_Sequences.html`.

[16] The mathlib Community, The Lean mathematical library, in *Certified Programs and Proofs (CPP 2020)*, ACM, 2020, 367–381.

[17] L. de Moura, S. Ullrich, The Lean 4 theorem prover and programming language, in *Automated Deduction (CADE 28)*, LNCS **12699**, Springer, 2021, 625–635.

[18] C. Marín, *Counting with the Derivative Tower: the Budan–Fourier Theorem, Machine-Checked in Lean 4* (2026, companion formalization; `BudanFourier.lean`).

[19] C. Marín, *The Staircase of Signs: Sturm's Root-Counting Theorem, Machine-Checked in Lean 4*, Zenodo, 2026, `doi:10.5281/zenodo.20707348`.

[20] C. Marín, *The Inward Bow of a Real-Rooted Polynomial: Newton's Inequalities, Machine-Checked in Lean 4*, Zenodo, 2026, `doi:10.5281/zenodo.20693064`.